



МИНОБРНАУКИ РОССИИ

Федеральное государственное бюджетное образовательное учреждение
высшего образования

«МИРЭА – Российский технологический университет»

РТУ МИРЭА

Институт искусственного интеллекта

Кафедра проблем управления

ПРАКТИЧЕСКАЯ РАБОТА №6

по дисциплине: **Информационные элементы робототехнических систем**

Тема лабораторной работы: «Обучение сегментатора на изображениях для определения масок объектов»

Студенты группы: КРБО-03-
23

Зенина А.А. _____

Грачев А.В. _____

Гришаев А.К. _____

Преподаватель:

ст. преподаватель Бычков А.М. _____

Работа представлена к
защите:

«___» _____ 2026 г.

Москва 2026

Сегментация фруктов и овощей

Этот ноутбук реализует полный пайплайн:

1. **Автоматическая генерация масок** из изображений на белом фоне
 2. **Обучение U-Net модели** на сгенерированных масках
 3. **Валидация и визуализация** результатов
 4. **Инференс** на произвольном изображении с диска
-

Установка зависимостей

```
import torch
```

```
print(f'PyTorch версия      : {torch.__version__}')
print(f'CUDA доступна      : {torch.cuda.is_available()}')
print(f'Кол-во GPU          : {torch.cuda.device_count()}')

if torch.cuda.is_available():
    print(f'Имя GPU           : {torch.cuda.get_device_name(0)}')
    print(f'Память GPU (МБ)      : {torch.cuda.get_device_properties(0).total_memory // 1024**2}')
else:
    print('\nПричина отсутствия CUDA:')
    print('  Проверьте torch.version.cuda :', torch.version.cuda)
```

```
PyTorch версия      : 2.12.0+cpu
CUDA доступна       : False
Кол-во GPU          : 0
```

```
Причина отсутствия CUDA:
  Проверьте torch.version.cuda : None
```

Импорты и глобальные настройки

```
import os
import cv2
import numpy as np
import torch
import torch.nn as nn
import torch.optim as optim
from torch.utils.data import Dataset, DataLoader
import torchvision.transforms.functional as TF
import albumentations as A
from albumentations.pytorch import ToTensorV2
from PIL import Image
import matplotlib.pyplot as plt
import matplotlib.patches as mpatches
from pathlib import Path
```

```
from tqdm import tqdm
import random
import warnings
warnings.filterwarnings('ignore')
```

— Воспроизводимость

```
SEED = 42
random.seed(SEED)
np.random.seed(SEED)
torch.manual_seed(SEED)
```

— Устройство

```
DEVICE = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
print(f'Используемое устройство: {DEVICE}')
```

— Гиперпараметры

```
IMG_SIZE      = 256    # размер входного изображения (пикселей)
BATCH_SIZE    = 8      # размер батча
NUM_EPOCHS    = 1      # количество эпох обучения
LR             = 1e-4   # скорость обучения
NUM_WORKERS   = 0      # потоки загрузки данных (0 = главный процесс)
```

— Пути к датасету

[!] УКАЖИТЕ СВОИ ПУТИ:

```
TRAIN_DIR =
r"C:\Users\admin_local\.cache\kagglehub\datasets\moltean\fruits\versions\87\fruits-360_100x100\fruits-360\Training" # например: r'C:/data/fruits/train'
VAL_DIR    =
r"C:\Users\admin_local\.cache\kagglehub\datasets\moltean\fruits\versions\87\fruits-360_100x100\fruits-360\Test" # например: r'C:/data/fruits/val'
```

```
print('Пути к датасету:')
print(f' Train : {TRAIN_DIR}')
print(f' Val   : {VAL_DIR}')
```

```
C:\Users\admin_local\AppData\Local\Packages\PythonSoftwareFoundation.Python.3
.12_qbz5n2kfra8p0\LocalCache\local-packages\Python312\site-
packages\albumintations\check_version.py:147: UserWarning: Error fetching
version info The read operation timed out
  data = fetch_version_info()
```

```
Используемое устройство: cpu
Пути к датасету:
```

```
Train :
C:\Users\admin_local\.cache\kagglehub\datasets\moltean\fruits\versions\87\fruits-360_100x100\fruits-360\Training
```

```
Val :  
C:\Users\admin_local\.cache\kagglehub\datasets\moltean\fruits\versions\87\fruits-360_100x100\fruits-360\Test
```

Блок генерации масок

Как это работает?

Поскольку датасет содержит **изображения на белом фоне**, маску объекта можно получить автоматически — без ручной разметки. Алгоритм состоит из нескольких шагов:

1. **Перевод в HSV** — цветовое пространство HSV позволяет легче отделить белый фон (высокая яркость $V \approx 255$, низкая насыщенность $S \approx 0$) от цветных объектов.
2. **Пороговая фильтрация** — пиксели с высокой яркостью ($V > 230$) и низкой насыщенностью ($S < 30$) считаются фоном; все остальные — объектом.
3. **Морфологические операции** — `closing` (дилатация → эрозия) закрывает мелкие дыры внутри объекта, `opening` убирает шум на фоне.
4. **Выбор наибольшего контура** — если на изображении несколько регионов, оставляем только самый большой (это и есть фрукт/овощ).
5. **Flood-fill от углов** — дополнительный шаг: заливаем фон начиная с угловых пикселей, чтобы убрать светлые артефакты по краям изображения.

В итоге получаем бинарную маску: **1 = объект, 0 = фон**.

```
#
```

```
# БЛОК: Генерация масок + предпросмотр
```

```
#
```

```
#
```

```
# КАК РАБОТАЕТ:
```

```
# 1. Переводим изображение в HSV
```

```
# 2. Белый фон → высокая яркость ( $V > 230$ ) + низкая насыщенность ( $S < 30$ )
```

```
# 3. Инвертируем → получаем маску объекта
```

```
# 4. Морфология: closing (заполняем дыры) + opening (убираем шум)
```

```
# 5. Flood-fill от углов → зачищаем светлые артефакты по краям
```

```
# 6. Оставляем только наибольший контур (сам фрукт/овощ)
```

```
#
```

```
#
```

```
IMG_EXTENSIONS = ('.jpg', '.jpeg', '.png', '.bmp', '.webp')
```

```
def generate_mask_from_white_bg(image_bgr: np.ndarray,
                                v_thresh: int = 230,
                                s_thresh: int = 30,
                                morph_ksize: int = 7) -> np.ndarray:
    """
    Генерирует бинарную маску объекта для изображения на белом фоне.

    Параметры
    -----
    image_bgr      : np.ndarray – входное изображение BGR (OpenCV)
    v_thresh       : int        – порог яркости V; пиксели выше считаются фоном
    s_thresh       : int        – порог насыщенности S; пиксели ниже = фон
    morph_ksize    : int        – размер ядра морфологических операций

    Возвращает
    -----
    mask : np.ndarray uint8, значения {0, 255} – 255=объект, 0=фон
    """
    hsv = cv2.cvtColor(image_bgr, cv2.COLOR_BGR2HSV)
    H, S, V = cv2.split(hsv)

    # Шаг 1: фон = светлый И ненасыщенный
    bg_mask = ((V > v_thresh) & (S < s_thresh)).astype(np.uint8) * 255
    obj_mask = cv2.bitwise_not(bg_mask)

    # Шаг 2: морфология
    kernel = cv2.getStructuringElement(cv2.MORPH_ELLIPSE, (morph_ksize,
morph_ksize))
    obj_mask = cv2.morphologyEx(obj_mask, cv2.MORPH_CLOSE, kernel,
iterations=3)
    obj_mask = cv2.morphologyEx(obj_mask, cv2.MORPH_OPEN, kernel,
iterations=1)

    # Шаг 3: flood-fill от угловых пикселей
    h, w = obj_mask.shape
    flood = obj_mask.copy()
    flood_mask = np.zeros((h + 2, w + 2), np.uint8)
    for pt in [(0, 0), (0, w - 1), (h - 1, 0), (h - 1, w - 1)]:
        if flood[pt] == 0:
            cv2.floodFill(flood, flood_mask, (pt[1], pt[0]), 128)
    flood[flood == 128] = 0
    obj_mask = flood

    # Шаг 4: оставляем только наибольший контур
    contours, _ = cv2.findContours(obj_mask, cv2.RETR_EXTERNAL,
```

```

cv2.CHAIN_APPROX_SIMPLE)
    if contours:
        largest = max(contours, key=cv2.contourArea)
        obj_mask = np.zeros_like(obj_mask)
        cv2.drawContours(obj_mask, [largest], -1, 255, thickness=cv2.FILLED)

    return obj_mask

```

```

def preview_masks(dataset_dir: str, n: int = 6):
    """Показывает n случайных пар (оригинал / маска) из датасета."""

```

— Диагностика пути

```

print(f'Путь          : {dataset_dir}')
print(f'Существует     : {os.path.exists(dataset_dir)}')

```

```

if not os.path.exists(dataset_dir):
    print('Папка не найдена! Проверьте переменную TRAIN_DIR.')
    return

```

```

top = os.listdir(dataset_dir)
print(f'Содержимое ({len(top)} эл.): {top[:10]}')

```

— Сбор изображений

```

all_images = []
for root, _, files in os.walk(dataset_dir):
    for f in files:
        if f.lower().endswith(IMG_EXTENSIONS):
            all_images.append(os.path.join(root, f))

```

```

print(f'Найдено изображений: {len(all_images)}')

```

```

if not all_images:
    print('\nИзображений не найдено!')
    print('Все файлы в папке (первые 30):')
    for root, _, files in os.walk(dataset_dir):
        for f in files[:30]:
            print(f'    {os.path.join(root, f)}')
    return

```

```

print(f'Пример: {all_images[0]}\n')

```

— Отрисовка

```

sample = random.sample(all_images, min(n, len(all_images)))
ncols = len(sample)

```

```

fig, axes = plt.subplots(2, ncols, figsize=(3 * ncols, 6))
if ncols == 1:
    axes = axes.reshape(2, 1) # защита если нашлось только 1 фото

fig.suptitle('Примеры автоматических масок', fontsize=14,
fontweight='bold')

for i, path in enumerate(sample):
    img_bgr = cv2.imread(path)
    if img_bgr is None:
        print(f'⚠ Не удалось прочитать: {path}')
        continue

    img_rgb = cv2.cvtColor(img_bgr, cv2.COLOR_BGR2RGB)
    mask = generate_mask_from_white_bg(img_bgr)

    # Оригинал
    axes[0, i].imshow(img_rgb)
    axes[0, i].set_title(Path(path).parent.name, fontsize=8,
fontweight='bold')
    axes[0, i].axis('off')

    # Оригинал + зелёная заливка объекта
    overlay = img_rgb.copy().astype(np.float32)
    green = np.array([0, 200, 50], dtype=np.float32)
    overlay[mask == 255] = overlay[mask == 255] * 0.5 + green * 0.5
    overlay = overlay.clip(0, 255).astype(np.uint8)

    axes[1, i].imshow(overlay)
    axes[1, i].set_title('маска', fontsize=8)
    axes[1, i].axis('off')

plt.tight_layout()
plt.show()
print('Предпросмотр готов.')

print('Функции генерации масок определены.')
print('Запускаем предпросмотр масок на тренировочных данных...\n')
preview_masks(TRAIN_DIR, n=6)

```

Функции генерации масок определены.
Запускаем предпросмотр масок на тренировочных данных...

```

Путь      :
C:\Users\admin_local\.cache\kagglehub\datasets\moltean\fruits\versions\87\fru
its-360_100x100\fruits-360\Training
Существует : True
Содержимое (257 эл.): ['Almonds 1', 'Apple 10', 'Apple 11', 'Apple 12',

```

```
'Apple 13', 'Apple 14', 'Apple 17', 'Apple 18', 'Apple 19', 'Apple 20']
```

Найдено изображений: 135071

Пример:

```
C:\Users\admin_local\.cache\kagglehub\datasets\moltean\fruits\versions\87\fruits-360_100x100\fruits-360\Training\Almonds 1\r0_0_100.jpg
```



Предпросмотр готов.

Dataset и аугментации

```
#
```

```
# Dataset: выбираем только нужные классы, макс. 75 изображений на класс
```

```
#
```

```
MAX_PER_CLASS = 75
```

```
# Классы которые хотим использовать – ищутся как подстрока в имени папки,  
# регистр не важен. Меняйте под свой датасет.
```

```
TARGET_FRUITS = ['banana', 'apple', 'pear', 'grape', 'pineapple']
```

```
def find_matching_classes(root_dir: str, targets: list) -> dict:
```

```
    """
```

```
    Ищет папки в root_dir, имя которых содержит одно из ключевых слов  
    targets.
```

```
    Возвращает словарь: {ключевое_слово: реальное_имя_папки}
```

```
    Например targets=['apple', 'banana'], папки ['Apple Red 1', 'Banana',  
    'Pear']
```

```
    вернёт {'apple': 'Apple Red 1', 'banana': 'Banana'}
```

```
    """
```

```
    all_dirs = [
```

```
        d for d in os.listdir(root_dir)
```

```
        if os.path.isdir(os.path.join(root_dir, d))
```



```

]

matched = {}
for target in targets:
    for d in all_dirs:
        if target.lower() in d.lower() and target not in matched:
            matched[target] = d
            break # берём первое совпадение для каждого ключевого слова

# Диагностика
print(f'\nПоиск классов в: {root_dir}')
for target in targets:
    if target in matched:
        print(f'  "{target}" → найдена папка: "{matched[target]}"')
    else:
        print(f'  "{target}" → папка НЕ найдена')

return matched

```

```

class FruitSegDataset(Dataset):
    IMG_EXTS = {' .jpg', ' .jpeg', ' .png', ' .bmp', ' .webp'}

    def __init__(self, root_dir: str, img_size: int = 256,
                 augment: bool = True,
                 target_fruits: list = TARGET_FRUITS,
                 max_per_class: int = MAX_PER_CLASS,
                 class_map: dict = None):
        """
        Параметры
        -----
        root_dir      : папка датасета (train или val)
        img_size      : размер входа модели
        augment       : аугментации (True для train, False для val)
        target_fruits : список ключевых слов для поиска классов
        max_per_class : максимум изображений из одного класса
        class_map     : готовый словарь {keyword: folder_name} –
                       передавайте из train_dataset.class_map в val,
                       чтобы гарантированно использовать те же классы
        """
        self.root_dir = root_dir
        self.img_size = img_size
        self.augment = augment

        # Находим папки или используем готовый маппинг
        if class_map is not None:
            self.class_map = {
                k: v for k, v in class_map.items()
                if os.path.isdir(os.path.join(root_dir, v))
            }

```

```

    }
    print(f'\nВал датасет использует те же классы что и трейн:')
    for k, v in self.class_map.items():
        exists = ' ' if os.path.isdir(os.path.join(root_dir, v)) else
'нет папки'
        print(f' {exists} "{k}" → "{v}"')
    else:
        self.class_map = find_matching_classes(root_dir, target_fruits)

    if not self.class_map:
        raise ValueError(
            f'Ни один из классов {target_fruits} не найден в
{root_dir}.\n'
            f'Папки в датасете: {os.listdir(root_dir)[:15]}'
        )

# Собираем изображения
self.image_paths = []
self.class_counts = {}

for keyword, folder_name in self.class_map.items():
    cls_dir = os.path.join(root_dir, folder_name)
    files = [
        os.path.join(cls_dir, f)
        for f in os.listdir(cls_dir)
        if Path(f).suffix.lower() in self.IMG_EXTS
    ]
    random.shuffle(files)
    files = files[:max_per_class]
    self.image_paths.extend(files)
    self.class_counts[folder_name] = len(files)

# Итоговая статистика
print(f'\nИтого классов : {len(self.class_map)}')
for folder, cnt in self.class_counts.items():
    print(f' {folder:<35} {cnt} изображений')
print(f'Всего          : {len(self.image_paths)} изображений')

# Аугментации
self.aug_transform =
    A.Compose([ A.Resize(img_size, img_size),
                A.HorizontalFlip(p=0.5),
                A.VerticalFlip(p=0.2),
                A.RandomRotate90(p=0.5),
                A.ShiftScaleRotate(shift_limit=0.1, scale_limit=0.2,
                                   rotate_limit=30, p=0.6,
                                   border_mode=cv2.BORDER_CONSTANT,
                                   value=(255, 255, 255)),
                A.ColorJitter(brightness=0.3, contrast=0.3,

```

```

        saturation=0.3, hue=0.1, p=0.5),
        A.GaussNoise(var_limit=(10, 50), p=0.3),
        A.Normalize(mean=(0.485, 0.456, 0.406),
                      std=(0.229, 0.224, 0.225)),
        ToTensorV2(),
    ])

    self.val_transform =
        A.Compose([ A.Resize(img_size, img_size),
                    A.Normalize(mean=(0.485, 0.456, 0.406),
                                std=(0.229, 0.224, 0.225)),
                    ToTensorV2(),
                ])

    def __len__(self):
        return len(self.image_paths)

    def __getitem__(self, idx):
        path = self.image_paths[idx]
        img_bgr = cv2.imread(path)
        if img_bgr is None:
            raise IOError(f'Не удалось прочитать: {path}')

        mask = generate_mask_from_white_bg(img_bgr)
        mask = (mask / 255).astype(np.uint8)
        img_rgb = cv2.cvtColor(img_bgr, cv2.COLOR_BGR2RGB)

        transform = self.aug_transform if self.augment else
self.val_transform
        result = transform(image=img_rgb, mask=mask)

        return result['image'], result['mask'].float()

# — Создаём датасеты

```

```

train_dataset = FruitSegDataset(TRAIN_DIR, img_size=IMG_SIZE, augment=True)

# val берёт ровно те же папки что нашёл train
val_dataset = FruitSegDataset(VAL_DIR, img_size=IMG_SIZE, augment=False,
                              class_map=train_dataset.class_map)

train_loader = DataLoader(train_dataset, batch_size=BATCH_SIZE,
                          shuffle=True, num_workers=NUM_WORKERS,
                          pin_memory=True)
val_loader = DataLoader(val_dataset, batch_size=BATCH_SIZE,
                        shuffle=False, num_workers=NUM_WORKERS,
                        pin_memory=True)

```

```
print(f'\nТренировочных батчей : {len(train_loader)}')
print(f'Валидационных батчей : {len(val_loader)}')
```

Поиск классов в:

C:\Users\admin_local\.cache\kagglehub\datasets\moltean\fruits\versions\87\fruits-360_100x100\fruits-360\Training

"banana" → найдена папка: "Banana 1"
"apple" → найдена папка: "Apple 10"
"pear" → найдена папка: "Pear 1"
"grape" → найдена папка: "Grape 1"
"pineapple" → найдена папка: "Pineapple 1"

Итого классов : 5

Banana 1	75 изображений
Apple 10	75 изображений
Pear 1	75 изображений
Grape 1	75 изображений
Pineapple 1	75 изображений

Всего : 375 изображений

Вал датасет использует те же классы что и трейн:

"banana" → "Banana 1"
"apple" → "Apple 10"
"pear" → "Pear 1"
"pineapple" → "Pineapple 1"

Итого классов : 4

Banana 1	75 изображений
Apple 10	75 изображений
Pear 1	75 изображений
Pineapple 1	75 изображений

Всего : 300 изображений

Тренировочных батчей : 47

Валидационных батчей : 38

Архитектура — U-Net

```
class DoubleConv(nn.Module):
```

```
    """Два последовательных свёрточных блока: Conv → BN → ReLU (x2)."""
```

```
    def __init__(self, in_ch: int, out_ch: int):
        super().__init__()
        self.block = nn.Sequential(
            nn.Conv2d(in_ch, out_ch, 3, padding=1, bias=False),
            nn.BatchNorm2d(out_ch),
            nn.ReLU(inplace=True),
            nn.Conv2d(out_ch, out_ch, 3, padding=1, bias=False),
            nn.BatchNorm2d(out_ch),
```

```

        nn.ReLU(inplace=True),
    )

    def forward(self, x):
        return self.block(x)

```

```
class UNet(nn.Module):
```

```
    """
```

```
    Классический U-Net для бинарной сегментации.
```

```
    Encoder: 4 уровня понижения разрешения (MaxPool + DoubleConv)
```

```
    Bottleneck: DoubleConv с максимальным числом каналов
```

```
    Decoder: 4 уровня с skip-connections (ConvTranspose2d + DoubleConv)
```

```
    Head: Conv 1x1 → 1 канал (Logit для бинарной маски)
```

```
    """
```

```
    def __init__(self, in_channels: int = 3, features=(64, 128, 256, 512)):
```

```
        super().__init__()
```

```
        self.downs = nn.ModuleList()
```

```
        self.ups = nn.ModuleList()
```

```
        self.pool = nn.MaxPool2d(2, 2)
```

```
    # — Encoder —
```

```
    ch = in_channels
```

```
    for f in features:
```

```
        self.downs.append(DoubleConv(ch, f))
```

```
        ch = f
```

```
    # — Bottleneck —
```

```
    self.bottleneck = DoubleConv(features[-1], features[-1] * 2)
```

```
    # — Decoder —
```

```
    for f in reversed(features):
```

```
        # Ансэмплинг
```

```
        self.ups.append(
```

```
            nn.ConvTranspose2d(f * 2, f, kernel_size=2, stride=2)
```

```
        )
```

```
        # DoubleConv после конкатенации со skip
```

```
        self.ups.append(DoubleConv(f * 2, f))
```

```
    # — Выходная голова —
```

```
    self.head = nn.Conv2d(features[0], 1, kernel_size=1)
```

```
    def forward(self, x):
```

```
        skips = []
```

```
        # Encoder
```

```
        for down in self.downs:
```

```

        x = down(x)
        skips.append(x)
        x = self.pool(x)

    x = self.bottleneck(x)
    skips = skips[::-1] # разворачиваем для decoder

    # Decoder
    for i in range(0, len(self.ups), 2):
        x = self.ups[i](x) # апсэмплинг
        skip = skips[i // 2]

        # Исправляем размер, если не совпадает
        if x.shape != skip.shape:
            x = TF.resize(x, skip.shape[2:])

        x = torch.cat([skip, x], dim=1) # skip-connection
        x = self.ups[i + 1](x) # DoubleConv

    return self.head(x) # raw logit [B, 1, H, W]

```

```

model = UNet().to(DEVICE)
total_params = sum(p.numel() for p in model.parameters() if p.requires_grad)
print(f'U-Net создан. Обучаемых параметров: {total_params:,}')

```

U-Net создан. Обучаемых параметров: 31,037,633

Функция потерь и метрики

```

class DiceBCELoss(nn.Module):
    """
    Комбинированная функция потерь: Dice Loss + Binary Cross-Entropy.

    Dice Loss хорошо работает при дисбалансе классов (фон >> объект),
    BCE обеспечивает стабильность градиентов на ранних этапах обучения.
    """

    def __init__(self, bce_weight: float = 0.5, smooth: float = 1e-6):
        super().__init__()
        self.bce_weight = bce_weight
        self.smooth = smooth
        self.bce = nn.BCEWithLogitsLoss()

    def forward(self, logits: torch.Tensor, targets: torch.Tensor):
        # BCE часть
        bce_loss = self.bce(logits, targets)

        # Dice часть
        probs = torch.sigmoid(logits)

```

```

        intersection = (probs * targets).sum(dim=(1, 2, 3))
        dice_loss = 1 - (2 * intersection + self.smooth) / (
            probs.sum(dim=(1, 2, 3)) + targets.sum(dim=(1, 2, 3)) +
self.smooth
        )
        dice_loss = dice_loss.mean()

    return self.bce_weight * bce_loss + (1 - self.bce_weight) * dice_loss

```

```

def dice_score(logits: torch.Tensor, targets: torch.Tensor,
               threshold: float = 0.5, smooth: float = 1e-6) -> float:
    """Вычисляет Dice coefficient (метрика качества, чем выше – тем
лучше)."""
    preds = (torch.sigmoid(logits) > threshold).float()
    intersection = (preds * targets).sum(dim=(1, 2, 3))
    score = (2 * intersection + smooth) / (
        preds.sum(dim=(1, 2, 3)) + targets.sum(dim=(1, 2, 3)) + smooth
    )
    return score.mean().item()

```

```

def iou_score(logits: torch.Tensor, targets: torch.Tensor,
               threshold: float = 0.5, smooth: float = 1e-6) -> float:
    """Вычисляет IoU / Jaccard index."""
    preds = (torch.sigmoid(logits) > threshold).float()
    intersection = (preds * targets).sum(dim=(1, 2, 3))
    union = preds.sum(dim=(1, 2, 3)) + targets.sum(dim=(1, 2, 3)) -
intersection
    score = (intersection + smooth) / (union + smooth)
    return score.mean().item()

```

```

criterion = DiceBCELoss(bce_weight=0.5)
optimizer = optim.AdamW(model.parameters(), lr=LR, weight_decay=1e-4)
scheduler = optim.lr_scheduler.CosineAnnealingLR(optimizer, T_max=NUM_EPOCHS,
eta_min=1e-6)

```

```
print('Loss, метрики, оптимизатор готовы.')
```

Loss, метрики, оптимизатор готовы.

Обучение модели

```

def train_one_epoch(model, loader, optimizer, criterion):
    model.train()
    total_loss, total_dice, total_iou = 0.0, 0.0, 0.0

    for images, masks in tqdm(loader, desc='Train', leave=False):
        images = images.to(DEVICE)

```

```

        masks = masks.unsqueeze(1).to(DEVICE)    # [B, H, W] → [B, 1, H, W]

        optimizer.zero_grad()
        logits = model(images)
        loss = criterion(logits, masks)
        loss.backward()
        optimizer.step()

        total_loss += loss.item()
        total_dice += dice_score(logits.detach(), masks)
        total_iou += iou_score(logits.detach(), masks)

    n = len(loader)
    return total_loss / n, total_dice / n, total_iou / n

@torch.no_grad()
def validate(model, loader, criterion):
    model.eval()
    total_loss, total_dice, total_iou = 0.0, 0.0, 0.0

    for images, masks in tqdm(loader, desc='Val ', leave=False):
        images = images.to(DEVICE)
        masks = masks.unsqueeze(1).to(DEVICE)

        logits = model(images)
        loss = criterion(logits, masks)

        total_loss += loss.item()
        total_dice += dice_score(logits, masks)
        total_iou += iou_score(logits, masks)

    n = len(loader)
    return total_loss / n, total_dice / n, total_iou / n

```

— Основной цикл обучения

```

history = {'train_loss': [], 'val_loss': [],
           'train_dice': [], 'val_dice': [],
           'train_iou': [], 'val_iou': []}

best_val_dice = 0.0
CHECKPOINT_PATH = 'best_unet.pth'

print(f'Начинаем обучение на {NUM_EPOCHS} эпох...\n')

for epoch in range(1, NUM_EPOCHS + 1):
    tr_loss, tr_dice, tr_iou = train_one_epoch(model, train_loader,

```



```

optimizer, criterion)
    vl_loss, vl_dice, vl_iou = validate(model, val_loader, criterion)
    scheduler.step()

    history['train_loss'].append(tr_loss)
    history['val_loss'].append(vl_loss)
    history['train_dice'].append(tr_dice)
    history['val_dice'].append(vl_dice)
    history['train_iou'].append(tr_iou)
    history['val_iou'].append(vl_iou)

    # Сохраняем лучшую модель
    if vl_dice > best_val_dice:
        best_val_dice = vl_dice
        torch.save(model.state_dict(), CHECKPOINT_PATH)
        flag = ' ← best'
    else:
        flag = ''

    print(f'Epoch [{epoch:02d}/{NUM_EPOCHS}]   '
          f'Train: loss={tr_loss:.4f} dice={tr_dice:.4f} iou={tr_iou:.4f} | '
          f'Val: loss={vl_loss:.4f} dice={vl_dice:.4f} '
          f'iou={vl_iou:.4f}{flag}')

print(f'\nОбучение завершено. Лучший Dice на val: {best_val_dice:.4f}')
print(f'Чекпоинт сохранён в: {CHECKPOINT_PATH}')

Начинаем обучение на 1 эпох...

```

```

Epoch [01/1] Train: loss=0.2814 dice=0.9266 iou=0.8703 | Val: loss=0.1489
dice=0.9638 iou=0.9304 ← best

```

```

Обучение завершено. Лучший Dice на val: 0.9638
Чекпоинт сохранён в: best_unet.pth

```

Графики обучения

```

epochs_range = range(1, NUM_EPOCHS + 1)

fig, axes = plt.subplots(1, 3, figsize=(18, 5))
fig.suptitle('История обучения', fontsize=14, fontweight='bold')

for ax, metric, title in
    zip(axes,
        [('train_loss', 'val_loss'), ('train_dice', 'val_dice'), ('train_iou',
        'val_iou')],
        ['Loss (DiceBCE)', 'Dice Score', 'IoU Score'])

```

```

):
    ax.plot(epochs_range, history[metric[0]], label='Train',
            color='steelblue', linewidth=2)
    ax.plot(epochs_range, history[metric[1]], label='Val', color='tomato',
            linewidth=2)
    ax.set_title(title, fontweight='bold')
    ax.set_xlabel('Эпоха')
    ax.legend()
    ax.grid(alpha=0.3)

plt.tight_layout()
plt.savefig('training_history.png', dpi=120, bbox_inches='tight')
plt.show()
print('График сохранён в training_history.png')

```

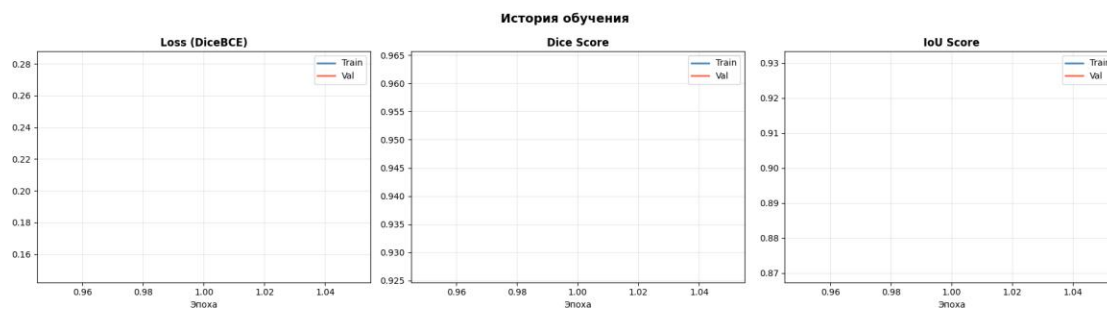


График сохранён в training_history.png

Визуализация результатов на валидационной выборке

Загружаем лучший чекпоинт

```

model.load_state_dict(torch.load(CHECKPOINT_PATH, map_location=DEVICE))
model.eval()

```

```

OVERLAY_COLOR = np.array([50, 205, 50], dtype=np.uint8)    # зелёный
OVERLAY_ALPHA = 0.45

```

ImageNet mean/std для денормализации

```

_MEAN = np.array([0.485, 0.456, 0.406])
_STD  = np.array([0.229, 0.224, 0.225])

```

```

def denormalize(tensor):

```

```

    """Tensor [3, H, W] → np.ndarray [H, W, 3] uint8."""

```

```

    img = tensor.cpu().numpy().transpose(1, 2, 0)

```

```

    img = (img * _STD + _MEAN).clip(0, 1)

```

```

    return (img * 255).astype(np.uint8)

```

```

@torch.no_grad()

```

```

def visualize_predictions(model, dataset, n: int = 8, threshold: float =
0.5):

```

```

    indices = random.sample(range(len(dataset)), min(n, len(dataset)))

```

```

fig, axes = plt.subplots(len(indices), 4,
                        figsize=(16, 4 * len(indices)))
if len(indices) == 1:
    axes = axes[np.newaxis, :]

cols = ['Оригинал', 'GT маска', 'Pred маска', 'Наложение']
for ax, col in zip(axes[0], cols):
    ax.set_title(col, fontweight='bold', fontsize=11)

for row, idx in enumerate(indices):
    image_t, mask_gt = dataset[idx]
    img_np = denormalize(image_t)

    logit = model(image_t.unsqueeze(0).to(DEVICE))
    pred_mask = (torch.sigmoid(logit) >
threshold).squeeze().cpu().numpy()
    gt_mask = mask_gt.numpy()

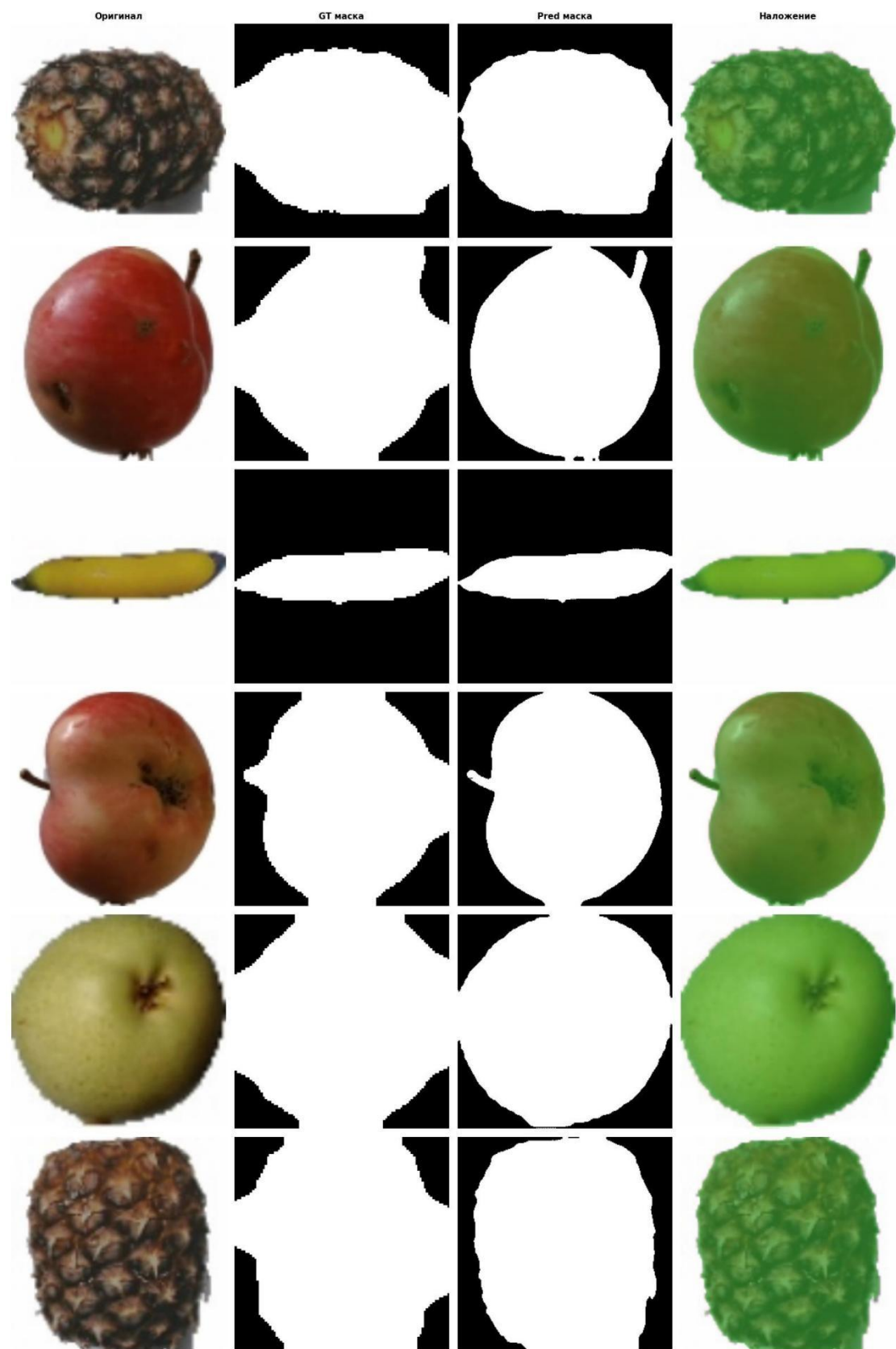
    # Наложение предсказанной маски
    overlay = img_np.copy()
    overlay[pred_mask == 1] = (
        img_np[pred_mask == 1] * (1 - OVERLAY_ALPHA)
        + OVERLAY_COLOR * OVERLAY_ALPHA
    ).astype(np.uint8)

    for col_ax, data, cmap in
        zip( axes[row],
            [img_np, gt_mask, pred_mask, overlay],
            [None, 'gray', 'gray', None]
        ):
        col_ax.imshow(data, cmap=cmap)
        col_ax.axis('off')

plt.tight_layout()
plt.savefig('val_predictions.png', dpi=120, bbox_inches='tight')
plt.show()
print('Визуализация сохранена в val_predictions.png')

visualize_predictions(model, val_dataset, n=6)

```



Визуализация сохранена в val_predictions.png

Инференс на произвольном изображении с диска

Укажите путь к изображению в переменной IMAGE_PATH и запустите блок.

```
# УКАЖИТЕ ПУТЬ К ВАШЕМУ ИЗОБРАЖЕНИЮ:
IMAGE_PATH = r"C:\Users\admin_local\Documents\AIR\AI_metod\pear.png"

#

```

```
MASK_THRESHOLD      = 0.5          # порог вероятности для бинаризации
CONTOUR_COLOR       = (0, 255, 0)  # BGR: зелёный контур
CONTOUR_THICKNESS   = 2
FILL_COLOR_RGB      = [50, 205, 50] # цвет заливки маски
FILL_ALPHA           = 0.4          # прозрачность заливки

_val_tf =
    A.Compose([ A.Resize(IMG_SIZE,
                        IMG_SIZE),
                A.Normalize(mean=(0.485, 0.456, 0.406), std=(0.229, 0.224, 0.225)),
                ToTensorV2(),
            ])

@torch.no_grad()
def predict_image(image_path: str,
                  threshold: float = MASK_THRESHOLD) -> dict:
    """
    Выполняет сегментацию одного изображения.

    Возвращает словарь:
        'original' : np.ndarray RGB оригинал
        'mask'      : np.ndarray бинарная маска {0,1}
        'overLay'   : np.ndarray RGB с цветной заливкой объекта
        'contoured' : np.ndarray RGB с контуром объекта
    """
    if not os.path.exists(image_path):
        raise FileNotFoundError(f'Файл не найден: {image_path}')

    img_bgr = cv2.imread(image_path)
    img_rgb = cv2.cvtColor(img_bgr, cv2.COLOR_BGR2RGB)
    h_orig, w_orig = img_rgb.shape[:2]

    # Преппроцессинг
    transformed = _val_tf(image=img_rgb)
    input_t = transformed['image'].unsqueeze(0).to(DEVICE)

    # Предсказание
    model.eval()
```

```

logit = model(input_t)                                # [1, 1, H, W]
prob = torch.sigmoid(logit).squeeze().cpu().numpy()    # [H, W] float
mask = (prob > threshold).astype(np.uint8)            # {0, 1}

# Возвращаем маску к оригинальному размеру
mask_full = cv2.resize(mask, (w_orig, h_orig),
                        interpolation=cv2.INTER_NEAREST)

# Цветная заливка объекта
overlay = img_rgb.copy().astype(np.float32)
fill = np.array(FILL_COLOR_RGB, dtype=np.float32)
obj_px = mask_full == 1
overlay[obj_px] = (overlay[obj_px] * (1 - FILL_ALPHA) + fill *
FILL_ALPHA)
overlay = overlay.clip(0, 255).astype(np.uint8)

# Контур объекта
contoured = img_rgb.copy()
mask_u8 = (mask_full * 255).astype(np.uint8)
contours, _ = cv2.findContours(mask_u8, cv2.RETR_EXTERNAL,
                               cv2.CHAIN_APPROX_SIMPLE)

cv2.drawContours(
    cv2.cvtColor(contoured, cv2.COLOR_RGB2BGR), # рисуем inplace в BGR
    contours, -1, CONTOUR_COLOR, CONTOUR_THICKNESS
)
# drawContours работает inplace по ссылке, но нужен BGR → обратно в RGB
contoured_bgr = cv2.cvtColor(img_rgb, cv2.COLOR_RGB2BGR)
cv2.drawContours(contoured_bgr, contours, -1, CONTOUR_COLOR,
CONTOUR_THICKNESS)
contoured = cv2.cvtColor(contoured_bgr, cv2.COLOR_BGR2RGB)

return {
    'original': img_rgb,
    'mask': mask_full,
    'overlay': overlay,
    'contoured': contoured,
}

```

```

def show_inference_result(result: dict, save_path: str =
'inference_result.png'):
    """Отображает результаты сегментации в виде 4 панелей."""
    titles = ['Оригинал', 'Бинарная маска', 'Заливка объекта', 'Контур
объекта']
    keys = ['original', 'mask', 'overlay', 'contoured']
    cmaps = [None, 'gray', None, None]

    fig, axes = plt.subplots(1, 4, figsize=(18, 5))
    fig.suptitle('Результат сегментации', fontsize=14, fontweight='bold')

```

```

for ax, key, title, cmap in zip(axes, keys, titles, cmaps):
    ax.imshow(result[key], cmap=cmap)
    ax.set_title(title, fontweight='bold')
    ax.axis('off')

# Легенда для маски
patch_obj = mpatches.Patch(color='white', label='Объект (1)')
patch_bg = mpatches.Patch(color='black', label='Фон (0)')
axes[1].legend(handles=[patch_obj, patch_bg],
               loc='lower right', fontsize=8,
               facecolor='dimgrey', labelcolor='white')

plt.tight_layout()
plt.savefig(save_path, dpi=150, bbox_inches='tight')
plt.show()
print(f'Результат сохранён в: {save_path}')

```

— Запуск инференса

```

result = predict_image(IMAGE_PATH, threshold=MASK_THRESHOLD)
show_inference_result(result)

print(f"\nСтатистика по маске:")
mask = result['mask']
total_px = mask.size
object_px = mask.sum()
print(f"  Всего пикселей      : {total_px:,}")
print(f"  Пикселей объекта     : {object_px:,} ({100*object_px/total_px:.1f}%)")
print(f"  Пикселей фона        : {total_px - object_px:,} ({100*(1-
object_px/total_px):.1f}%)")

```



Результат сохранён в: inference_result.png

Статистика по маске:

```

Всего пикселей      : 350,701
Пикселей объекта    : 200,369 (57.1%)
Пикселей фона       : 150,332 (42.9%)

```

Сохранение / загрузка модели

```
# Сохранить модель вручную  
torch.save(model.state_dict(), 'UNET_fruit_seg.pth')  
print('Модель сохранена в UNET_fruit_seg.pth')  
  
# Загрузить позже:  
# model_loaded = UNet().to(DEVICE)  
# model_loaded.load_state_dict(torch.load('UNET_fruit_seg.pth',  
map_location=DEVICE))  
# model_loaded.eval()
```

Модель сохранена в UNET_fruit_seg.pth